

Guía de Estudio — Tema 04

Aspectos Avanzados de Programación Funcional

Materia: Paradigmas y Lenguajes de Programación 2026 — UNTDF / IDEI **Clase:** 04 | **Duración**
estimada de lectura: 5-6 horas **Filminas de referencia:** F-01 a F-36

Propósito

Esta guía es tu compañero de estudio autónomo para el Tema 04. Está organizada siguiendo la misma estructura de las filminas de clase, pero con **desarrollo teórico expandido**, **ejemplos paso a paso** y **ejercicios resueltos** para que puedas entender cada concepto en profundidad. No necesitás haber visto la clase para aprovecharla — todo está autocontenido.

Objetivos de aprendizaje

Al terminar esta guía, vas a poder:

1. Explicar qué son las funciones de primera clase y las funciones de orden superior.
2. Construir pipelines de transformación de datos con `pipe` (TypeScript) y `->>` (Clojure).
3. Usar `map`, `filter`, `flatMap` y `reduce` para transformar colecciones sin mutación.
4. Diferenciar partial application de currying y aplicar ambos en código real.
5. Modelar errores explícitamente con `Result<T, E>` en lugar de excepciones.
6. Construir middleware composable como funciones de orden superior.
7. Implementar recursión de cola con acumuladores en Clojure y TypeScript.
8. Entender memoization, lazy sequences y el patrón data-driven de Clojure.

Conceptos previos necesarios

- Funciones puras e inmutabilidad (Tema 03).
 - Tipos básicos de TypeScript: generics, union types, type aliases.
 - Sintaxis elemental de Clojure: `defn`, `let`, `if`, listas y mapas.
 - Uso básico de `map`, `filter`, `reduce` (vistos en Tema 03).
-

Parte 1 — Funciones de orden superior y composición

Filminas de referencia: F-04 a F-12

1.1 Funciones como valores de primera clase

En programación funcional, una función tiene el mismo estatus que un número, un string o un objeto. Se puede:

- **Asignar a una variable:** `const doble = (x: number) => x * 2;`
- **Pasar como argumento:** `[1, 2, 3].map(doble)`
- **Devolver como resultado:** una función que crea otra función

Esto se llama **first-class functions** y es la base de todo lo que viene después. Si las funciones no fueran valores, no podría existir `map`, `filter`, `compose` ni ningún patrón avanzado.

Ejemplo paso a paso:

```
// 1. Asignar una función a una variable
const saludar = (nombre: string) => `Hola, ${nombre}!`;

// 2. Pasar una función como argumento
const nombres = ["Ana", "Luis", "María"];
const saludos = nombres.map(saludar);
// → ["Hola, Ana!", "Hola, Luis!", "Hola, María!"]

// 3. Devolver una función como resultado
const crearSaludo = (prefijo: string) =>
  (nombre: string) => `${prefijo}, ${nombre}!`;

const saludoFormal = crearSaludo("Estimado/a");
saludoFormal("Ana"); // → "Estimado/a, Ana!"
```

Contraste imperativo vs funcional:

```
// Imperativo: digo CÓMO iterar
const resultados: string[] = [];
for (let i = 0; i < nombres.length; i++) {
  resultados.push(saludar(nombres[i]));
}

// Funcional: digo QUÉ hacer con cada elemento
const resultados = nombres.map(saludar);
```

La versión funcional es más corta, no tiene estado mutable (`resultados` con `push`) y declara la intención directamente.

1.2 Higher-Order Functions (HOF)

Una **función de orden superior** es una función que recibe y/o devuelve otra función. Las 5 HOF fundamentales son:

HOF	Qué hace	Firma simplificada
<code>map</code>	Aplica una función a cada elemento	$(A \rightarrow B, [A]) \rightarrow [B]$
<code>filter</code>	Selecciona elementos que cumplen un predicado	$(A \rightarrow \text{bool}, [A]) \rightarrow [A]$
<code>reduce</code>	Pliega una colección a un valor	$((\text{acc}, A) \rightarrow \text{acc}, \text{init}, [A]) \rightarrow \text{acc}$
<code>compose</code>	Combina funciones de derecha a izquierda	$(B \rightarrow C, A \rightarrow B) \rightarrow (A \rightarrow C)$
<code>pipe</code>	Combina funciones de izquierda a derecha	$(A \rightarrow B, B \rightarrow C) \rightarrow (A \rightarrow C)$

Estas 5 funciones son el **vocabulario** con el que se construyen todos los patrones avanzados de esta guía. Si las dominás, el resto es combinación.

1.3 Composición de funciones: `compose` y `pipe`

Componer funciones significa encadenarlas para que la salida de una sea la entrada de la siguiente.

`compose` aplica de derecha a izquierda (como en matemáticas): $\text{compose}(f, g, h)(x) = f(g(h(x)))$

`pipe` aplica de izquierda a derecha (más legible para programadores): $\text{pipe}(h, g, f)(x) = f(g(h(x)))$

Reglas fundamentales de la composición:

1. Cada función recibe el output de la anterior.
2. Los tipos deben ser compatibles entre funciones consecutivas.
3. El resultado es una **nueva función** — no se ejecuta nada hasta que se llame.

Ejemplo resuelto paso a paso en TypeScript:

```
// Implementación de pipe
const pipe = <T>(...fns: Array<(x: T) => T>) =>
  (x: T): T => fns.reduce((acc, fn) => fn(acc), x);

// Funciones unitarias
const trim = (s: string) => s.trim();
const lowercase = (s: string) => s.toLowerCase();
const addDomain = (s: string) =>
  s.includes("@") ? s : `${s}@empresa.com`;

// Pipeline nombrado
const normalizeEmail = pipe(trim, lowercase, addDomain);

// Trazar la ejecución paso a paso:
// Input: " ANA "
// 1. trim(" ANA ")    → "ANA"
// 2. lowercase("ANA")  → "ana"
// 3. addDomain("ana")  → "ana@empresa.com"

normalizeEmail(" ANA "); // → "ana@empresa.com"
```

¿Cómo funciona **`pipe`** internamente? Recibe un array de funciones y devuelve una nueva función. Cuando esa nueva función se llama con un valor, usa `reduce` para aplicar cada función en secuencia, pasando el resultado de una a la siguiente.

1.4 Thread macro `->>` en Clojure

Clojure tiene un operador nativo que hace lo mismo que `pipe`: el thread macro `->>`.

```
;; Sin ->> (difícil de leer — se lee de adentro hacia afuera)
(reduce + (map #(* % %) (filter even? [1 2 3 4 5])))

;; Con ->> (fluye de arriba hacia abajo)
(->> [1 2 3 4 5]
  (filter even?)      ; → (2 4)
  (map #(* % %))      ; → (4 16)
  (reduce +))         ; → 20
```

Traza paso a paso:

1. Empezamos con `[1 2 3 4 5]`
2. `(filter even? [1 2 3 4 5])` → `(2 4)` — solo los pares
3. `(map #(* % %) (2 4))` → `(4 16)` — cada uno al cuadrado
4. `(reduce + (4 16))` → `20` — sumar todo

`->>` inserta el resultado anterior como **último argumento** de cada forma. Es exactamente la misma idea que `pipe` en TypeScript.

Ejemplo práctico: suma de pedidos completados

```
(->> orders
  (filter #(= (:status %) :completed)) ; pedidos completados
  (map :total)                          ; extraer totales
  (reduce + 0))                        ; sumar
```

1.5 Inmutabilidad práctica

En FP, nunca modificamos un dato existente — creamos uno nuevo.

```
// ❌ Mutación directa (antipatrón)
function normalize(user: User): void {
  user.email = user.email.trim().toLowerCase(); // modifica el original
}

// ✅ Función pura (crea nuevo objeto)
const trimUser = (u: User): User =>
  ({ ...u, email: u.email.trim(), name: u.name.trim() });

const lowercaseEmail = (u: User): User =>
  ({ ...u, email: u.email.toLowerCase() });

// Composición: pipeline de normalización
const normalizeUser = pipe(trimUser, lowercaseEmail);

const raw = { name: " Ana ", email: " ANA@test.com " };
const clean = normalizeUser(raw);
// raw sigue igual: { name: " Ana ", email: " ANA@test.com " }
// clean es nuevo: { name: "Ana", email: "ana@test.com" }
```

El operador spread `{ ...u, key: newVal }` crea un nuevo objeto con todas las propiedades de `u` más la propiedad reemplazada. El original queda intacto.

¿Por qué importa?

- **Debugging:** podés comparar estados anteriores y actuales porque ambos existen.
- **React/Vue:** la comparación por referencia detecta cambios eficientemente (`raw !== clean`).
- **Concurrencia:** sin mutación, no hay race conditions.

1.6 `map`, `filter`, `flatMap` y `reduce` en profundidad

Estas son las 4 operaciones declarativas sobre colecciones:

`map` — Transformar cada elemento

```
type User = { name: string; email: string };

const users: User[] = [
  { name: "Ana", email: "ana@test.com" },
  { name: "Luis", email: "luis@test.com" },
];

// De usuarios a emails
const emails = users.map(u => u.email);
// → ["ana@test.com", "luis@test.com"]
```

Firma: `Array<A>.map(A → B): Array<B>` — transforma cada elemento de tipo A a tipo B. La cantidad de elementos NO cambia.

`filter` — Seleccionar un subconjunto

```
const activos = users.filter(u => u.active);
// Solo los usuarios activos. El tipo no cambia: sigue siendo User[].
```

Firma: `Array<A>.filter(A → boolean): Array<A>` — selecciona elementos. El tipo NO cambia, la cantidad puede reducirse.

`flatMap` — Transformar Y aplanar

```
type User = { name: string; roles: string[] };

const users: User[] = [
  { name: "Ana", roles: ["admin", "editor"] },
  { name: "Luis", roles: ["editor"] },
  { name: "María", roles: ["viewer", "editor"] },
];

// Lista plana de todos los roles
const allRoles = users.flatMap(u => u.roles);
// → ["admin", "editor", "editor", "viewer", "editor"]

// Roles únicos
const uniqueRoles = [...new Set(users.flatMap(u => u.roles))];
// → ["admin", "editor", "viewer"]
```

Firma: `Array<A>.flatMap(A → Array<B>): Array<B>` — cada elemento produce un array, y `flatMap` aplanar todo en un solo nivel. Sin `flatMap`, necesitarías `.map(...).flat()`.

Equivalente Clojure: `(mapcat :roles users)` — `mapcat` = `map` + `concat` .

`reduce` — El patrón más general

`reduce` “pliega” una colección en cualquier tipo de resultado. Es tan general que `map` y `filter` son casos especiales de `reduce` .

```
// Ejemplo 1: sumar revenues
const totalRevenue = orders
  .filter(o => o.status === "completed")
  .map(o => o.amount)
  .reduce((sum, amt) => sum + amt, 0);

// Ejemplo 2: construir un índice (diccionario)
const userById = users.reduce(
  (index, user) => ({ ...index, [user.id]: user }),
  {} as Record<string, User>
);
// De [{ id: "1", name: "Ana" }, ...] a { "1": { id: "1", name: "Ana" }, ... }
```

Equivalente Clojure con `->>` :

```
(->> orders
  (filter #(= :completed (:status %)))
  (map :amount)
  (reduce + 0))
```

Parte 2 — Aplicación parcial, currying y validación web

Filminas de referencia: F-13 a F-23

2.1 Partial application

Aplicación parcial significa fijar algunos argumentos de una función para obtener una función más específica.

```
add(a, b) = a + b           // función de 2 argumentos
add5 = partial(add, 5)      // función de 1 argumento (a=5 ya está fijado)
add5(3) → 8
```

La función parcial **NO** se ejecuta — devuelve una nueva función que espera los argumentos que faltan.

Ejemplo práctico en TypeScript — fábricas de validadores:

```
// Función genérica de validación "requiere campo"
const makeRequiredValidator =
  (fieldName: string) => // arg fijado
  (value: string): Result<string, string> => // arg pendiente
    value.trim()
      ? { status: "ok", value }
      : { status: "error", error: `${fieldName} es obligatorio` };

// Creamos validadores específicos (partial application)
const validateName = makeRequiredValidator("nombre");
const validateEmail = makeRequiredValidator("email");

// Uso:
validateName("Ana"); // → { status: "ok", value: "Ana" }
validateName(""); // → { status: "error", error: "nombre es obligatorio" }
```

Cada validator es **stateless** (no tiene estado interno) y **testable** por separado.

Equivalente en Clojure — **partial** es nativo:

```
(defn multiply [factor n] (* factor n))

(def double (partial multiply 2)) ; fix factor=2
(def triple (partial multiply 3)) ; fix factor=3

(map double [1 2 3 4]) ;→ (2 4 6 8)
(map triple [1 2 3 4]) ;→ (3 6 9 12)
```

2.2 Currying

Currying transforma una función de N argumentos en N funciones anidadas de 1 argumento.

$f(a, b, c) \rightarrow \text{curry}(f) = a \rightarrow b \rightarrow c \rightarrow \text{resultado}$

Diferencia clave con partial application:

	Partial Application	Currying
Qué hace	Fija <i>algunos</i> argumentos	Convierte en cadena de 1 argumento
Cuándo aplica	En el momento de usar	Al definir la función
Flexibilidad	Cualquier cantidad de args	Siempre 1 arg por vez

¿Por qué importa? Currying permite construir variantes de funciones sin duplicar código y habilita composición cuando **pipe** / **compose** necesitan funciones de 1 argumento.

Ejemplo resuelto paso a paso en TypeScript:

```
// Paso 1: curry genérico para funciones de 2 argumentos
const curry2 = <A, B, C>{fn: (a: A, b: B) => C} =>
  (a: A) => (b: B): C => fn(a, b);

// Paso 2: función original de 2 argumentos
const hasMinLength = (min: number, str: string) => str.length >= min;

// Paso 3: versión currificada
const cHasMinLength = curry2(hasMinLength);

// Paso 4: crear validadores especializados
const atLeast3 = cHasMinLength(3); // para nombre
const atLeast8 = cHasMinLength(8); // para contraseña

// Paso 5: usar en pipeline
atLeast3("Ana"); // → true
atLeast8("abc"); // → false (solo 3 chars)
atLeast8("password123"); // → true

// Paso 6: combinar en un pipeline de validación
const validatePassword = pipe(
  trim,
  (s: string) => atLeast8(s)
  ? { status: "ok" as const, value: s }
  : { status: "error" as const, error: "mínimo 8 caracteres" }
);
```

Equivalente en Clojure — con lambdas anidadas:

```
;; "Currying" idiomático en Clojure: funciones que devuelven funciones
(defn make-validator [pred error-msg]
  (fn [value]
    (if (pred value)
      {:status :ok :value value}
      {:status :error :error error-msg})))

;; Validators especializados
(def validate-email
  (make-validator
    #(re-matches #" Cant @ Cant " %)
    "email inválido"))

(def validate-not-empty
  (make-validator
    #(seq (clojure.string/trim %))
    "campo vacío"))

(validate-email "ana@test.com") ;> {:status :ok, :value "ana@test.com"}
(validate-email "invalido") ;> {:status :error, :error "email inválido"}
```

2.3 El tipo `Result<T, E>` — errores sin excepciones

Las excepciones rompen el flujo funcional: son un `goto` encubierto. El tipo `Result` modela el éxito o el fracaso como un valor.

```
type Result<T, E> =
  | { status: "ok"; value: T }
  | { status: "error"; error: E };
```

Ventajas sobre try/catch:

1. **Explícito**: el tipo de retorno dice que puede fallar — no hay sorpresas.
2. **Forzado por el compilador**: TypeScript obliga a manejar ambos casos.
3. **Composable**: se puede encadenar con `chain` o `andThen`.
4. **Testeable**: no necesitas `expect(...).toThrow()`.

Regla de oro: si una función puede fallar, su tipo de retorno debe decirlo.

2.4 Validación encadenada con `Result` — ejemplo completo

Este es el ejemplo más importante de la clase. Trazamos paso a paso cómo construir un pipeline de validación completo:

```
// Paso 1: definir los tipos
type FormData = { name: string; email: string; password: string };
type Validator<T> = (data: T) => Result<T, string>;

// Paso 2: crear validadores individuales (cada uno hace UNA cosa)
const requireName: Validator<FormData> =
  data => data.name.trim()
    ? { status: "ok", value: data }
    : { status: "error", error: "nombre requerido" };

const requireValidEmail: Validator<FormData> =
  data => /^.+@.+\.+$.test(data.email)
    ? { status: "ok", value: data }
    : { status: "error", error: "email inválido" };

const requireStrongPassword: Validator<FormData> =
  data => data.password.length >= 8
    ? { status: "ok", value: data }
    : { status: "error", error: "contraseña muy corta" };

// Paso 3: función de encadenamiento
// Si hay error → propaga. Si hay ok → continúa.
const chain = <T>({
  result: Result<T, string>,
  validator: Validator<T>
}): Result<T, string> =>
  result.status === "error" ? result : validator(result.value);

// Paso 4: combinar todos los validadores
const validateForm = (data: FormData): Result<FormData, string> =>
  [requireName, requireValidEmail, requireStrongPassword]
    .reduce(chain, { status: "ok", value: data });
```

Traza con datos válidos:

```
Input: { name: "Ana", email: "ana@test.com", password: "12345678" }
1. requireName → { status: "ok", value: data }   ✅ pasa
2. requireValidEmail → { status: "ok", value: data }   ✅ pasa
3. requireStrongPassword → { status: "ok", value: data }   ✅ pasa
→ Resultado final: { status: "ok", value: data }
```

Traza con error:

```
Input: { name: "Ana", email: "invalido", password: "12345678" }
1. requireName → { status: "ok", value: data }   ✅ pasa
2. requireValidEmail → { status: "error", error: "email inválido" }   ❌ falla
3. (se saltea — chain propaga el error)
→ Resultado final: { status: "error", error: "email inválido" }
```

2.5 Result en un handler HTTP

```
const registerHandler = async (req: Request, res: Response) => {
  const result = validateForm(req.body);

  if (result.status === "error") {
    return res.status(400).json({ error: result.error });
  }

  // TypeScript sabe que result.value es FormData válido
  const user = await userService.create(result.value);
  return res.status(201).json(user);
};
```

No hay `try/catch`. El flujo es lineal y explícito. El tipo `Result` actúa como **protocolo** entre la validación y la lógica de negocio.

2.6 Middleware composable

Un **middleware** es una función que recibe un handler y devuelve un handler transformado:

$$\\text{Middleware} = (\\text{Request} \\rightarrow \\text{Response}) \\rightarrow (\\text{Request} \\rightarrow \\text{Response})$$

Esto es una HOF: recibe y devuelve funciones.

Ejemplo completo:

```
type Request = { headers: Record<string, string>; body: unknown };
type Response = { status: number; body: unknown };
type Handler = (req: Request) => Promise<Response>;
type Middleware = (handler: Handler) => Handler;

// Middleware de autenticación (usa partial application para fijar el secreto)
const withAuth = (secret: string): Middleware =>
  handler => async req => {
    const token = req.headers["authorization"];
    if (token !== `Bearer ${secret}`) {
      return { status: 401, body: { error: "unauthorized" } };
    }
    return handler(req);
  };

// Middleware de logging (usa partial application para fijar el prefijo)
const withLogging = (prefix: string): Middleware =>
  handler => async req => {
    console.log(`[${prefix}] incoming request`);
    const res = await handler(req);
    console.log(`[${prefix}] response ${res.status}`);
    return res;
  };

// Composición: aplicar middlewares con pipe
const secured = pipe(
  withLogging("auth-route"),
  withAuth("my-secret")
)(baseHandler);
```

`withAuth("my-secret")` es **partial application**: fija el secreto y devuelve un middleware genérico. Lo mismo para `withLogging("auth-route")`.

Parte 3 — Recursión de cola y patrones avanzados

Filminas de referencia: F-24 a F-31

3.1 El problema de la recursión simple

La recursión simple acumula frames en el call stack:

```
factorial(5)
→ 5 * factorial(4)      // frame 1 - espera
  → 4 * factorial(3)    // frame 2 - espera
    → 3 * factorial(2)  // frame 3 - espera
      → 2 * factorial(1) // frame 4 - espera
        → 1             // resuelve
        ← 2 * 1 = 2
      ← 3 * 2 = 6
    ← 4 * 6 = 24
  ← 5 * 24 = 120
```

Cada llamada queda **pendiente** hasta que la interna termine. Con listas de 10.000 o 100.000 elementos → **stack overflow**.

3.2 Recursión de cola (Tail Call Optimization)

Una llamada es **de cola** cuando es la **última operación** de la función — no hay trabajo pendiente después.

```
// NO es tail call: queda la multiplicación pendiente
factorial(n) = n * factorial(n-1)

// Sí es tail call: el acumulador lleva el resultado parcial
factorial(n, acc=1) = factorial(n-1, acc*n)    // nada pendiente
```

Con un acumulador, el runtime puede **reemplazar** el frame actual en lugar de crear uno nuevo → no hay acumulación de stack.

3.3 **recur** en Clojure

Clojure **garantiza** TCO con la palabra clave **recur**. Si la llamada no está en posición de cola, Clojure da error de compilación.

```
;; Suma de lista con recur
(defn sum-list [nums acc]
  (if (empty? nums)
      acc
      (recur (rest nums) (+ acc (first nums)))))
```

Traza paso a paso:

```
(sum-list [1 2 3] 0)
→ nums=[1 2 3], acc=0. No vacío → recur [2 3] (0+1)
→ nums=[2 3],   acc=1. No vacío → recur [3]   (1+2)
→ nums=[3],     acc=3. No vacío → recur []    (3+3)
→ nums=[],      acc=6. Vacío   → devuelve 6
```

No se acumulan frames: cada `recur` reemplaza el frame actual.

Ejemplo más complejo — flatten:

```
(defn my-flatten [xs acc]
  (cond
    (empty? xs)      (reverse acc)
    (seq? (first xs)) (recur (concat (first xs) (rest xs)) acc)
    :else            (recur (rest xs) (cons (first xs) acc))))

(my-flatten [1 [2 3] [4 [5 6]] []])
;→ (1 2 3 4 5 6)
```

3.4 Recursión de cola en TypeScript

JavaScript/TypeScript **NO** garantizan TCO (V8/Chrome/Node no lo implementan). Pero la lógica del acumulador sigue siendo útil:

```
// Suma con acumulador
const sumList = (nums: number[], acc = 0): number =>
  nums.length === 0
    ? acc
    : sumList(nums.slice(1), acc + nums[0]);

// Búsqueda en árbol con stack explícito
type TreeNode = { value: number; children: TreeNode[] };

const findInTree = (
  nodes: TreeNode[],
  target: number
): number | null => {
  if (nodes.length === 0) return null;
  const [head, ...tail] = nodes;
  if (head.value === target) return head.value;
  return findInTree([...head.children, ...tail], target);
};
```

Si el stack puede ser muy profundo (miles de niveles), convertir a loop con stack explícito o usar trampolining.

3.5 Memoization — cache funcional

Memoization guarda el resultado de una función para no recalcularlo cuando se llama con los mismos argumentos.

```
memoize(f)(x)
→ si x ya fue calculado: devuelve cache[x]    O(1)
→ si no: calcula f(x), guarda en cache, devuelve
```

Cuándo usar: funciones puras con resultados costosos y llamadas repetidas con los mismos valores.

Cuándo NO usar: funciones con efectos colaterales (la cache podría devolver datos stale).

Implementación en TypeScript:

```
const memoize = <T, R>(fn: (arg: T) => R): ((arg: T) => R) => {
  const cache = new Map<T, R>();
  return (arg: T): R => {
    if (cache.has(arg)) return cache.get(arg)!;
    const result = fn(arg);
    cache.set(arg, result);
    return result;
  };
};

// Caso real: parseo de configuración (costoso)
const getConfig = memoize((env: string) => parseEnvConfig(env));
getConfig("production"); // parseEnvConfig corre una vez
getConfig("production"); // desde cache – instantáneo
```

En Clojure, `memoize` es nativo:

```
(defn fetch-user [id]
  (Thread/sleep 100) ; simula latencia de red
  {:id id :name (str "User-" id)})

(def fetch-user-cached (memoize fetch-user))

(fetch-user-cached 42) ; llama a fetch-user (100ms)
(fetch-user-cached 42) ; desde cache (0ms)
```

3.6 Lazy sequences en Clojure

Una **lazy sequence** no calcula sus elementos hasta que se necesitan. Esto permite modelar secuencias **potencialmente infinitas** sin explotar la memoria.

```
;; range infinita – no explota memoria
(def naturals (range)) ; 0, 1, 2, 3, ... infinito

;; Tomar solo lo que necesito
(take 5 naturals) ;> (0 1 2 3 4)
(take 5 (filter even? naturals)) ;> (0 2 4 6 8)
```

Ejemplo práctico — streaming de logs:

```
;; Procesar un stream de logs (potencialmente infinito)
(->> (read-log-stream)
  (filter #(= :error (:level %))) ; solo errores
  (map :message) ; extraer mensaje
  (take 100)) ; primeros 100

;; Sin lazy: cargar TODOS los logs en memoria antes de filtrar
;; Con lazy: procesar 1 elemento a la vez hasta completar take
```

Equivalente conceptual en TypeScript: generators (`function*`).

3.7 Mini-DSLs data-driven en Clojure

Un patrón muy poderoso en Clojure es expresar las reglas como **datos** en lugar de código:

```
;; Las reglas son datos – vectores de mapas
(def user-rules
  [{:field :name :pred #(seq %) :msg "nombre requerido"}
   {:field :email :pred #(re-matches #" Cant match email") :msg "email inválido"}
   {:field :age :pred #(>= % 18) :msg "debe ser mayor de edad"}])

;; Motor genérico – aplica cualquier conjunto de reglas a cualquier mapa
(defn validate [rules data]
  (->> rules
    (map (fn [{:keys [field pred msg]}]
          (when-not (pred (get data field))
            {:field field :error msg})))
    (remove nil?)))

;; Uso
(validate user-rules {:name "" :email "x" :age 16})
;; → [{:field :name, :error "nombre requerido"}
;;     {:field :email, :error "email inválido"}
;;     {:field :age, :error "debe ser mayor de edad"}]
```

¿Por qué es poderoso?

- Las reglas se pueden **serializar** (guardar en un archivo, base de datos).
- Se pueden **modificar en runtime** sin recompilar.
- El motor de validación es **genérico** — sirve para cualquier entidad.
- Es el patrón **data-driven programming** — la lógica vive en los datos.

Parte 4 — Comparación y aplicación

Filminas de referencia: F-32 a F-36

4.1 Tabla comparativa Clojure ↔ TypeScript

Patrón	Clojure	TypeScript
Pipeline	<code>->></code> thread macro	<code>pipe(...fns)</code>
Map	<code>(map f xs)</code>	<code>xs.map(f)</code>
Filter	<code>(filter pred xs)</code>	<code>xs.filter(pred)</code>
Reduce	<code>(reduce f acc xs)</code>	<code>xs.reduce(f, acc)</code>
Partial	<code>(partial f a)</code>	<code>(b) => f(a, b)</code> (closure)
Lazy	<code>(range)</code> , <code>(lazy-seq ...)</code>	<code>function*</code> (generators)
TCO	<code>recur</code> (garantizado)	Manual / trampolining
Datos	Mapas, vectores, listas	Objetos, arrays

Clave: los patrones son **transferibles**. Si entendés uno, el otro es adaptación de sintaxis. Clojure aporta claridad conceptual; TypeScript aporta tipado estático.

4.2 Guía práctica: cuándo usar cada patrón

Usá FP cuando:

- Necesitás transformar datos sin mutar el original → HOF, `map/filter/reduce`.
- Tenés lógica que puede fallar → `Result` en lugar de `throw`.
- Querés construir handlers/validators configurables → `partial`, `curry`.
- Necesitás encadenar operaciones sobre datos → `pipe`.
- Una función costosa se llama muchas veces con los mismos args → `memoize`.
- Procesás streams grandes → lazy sequences.

No forzar FP cuando:

- El estado mutable local es claro y acotado (ej: buffer interno de una clase).
- La lógica es UI-event-heavy y el modelo reactivo es más natural.

Ejercicios

Ejercicio 1 — Pipeline de transformación

Dado este array de productos:

```
type Product = { name: string; price: number; category: string; inStock: boolean };

const products: Product[] = [
  { name: "Teclado",    price: 25000, category: "periféricos", inStock: true },
  { name: "Monitor",   price: 85000, category: "periféricos", inStock: true },
  { name: "Auriculares", price: 12000, category: "audio",      inStock: false },
  { name: "Micrófono",  price: 18000, category: "audio",      inStock: true },
  { name: "Mouse",     price: 8000,  category: "periféricos", inStock: true },
];
```

Consigna:

1. Filtrar los productos en stock.
2. Filtrar solo los de la categoría "periféricos".
3. Extraer los precios.
4. Calcular el total.
5. Escribirlo como un pipeline con encadenamiento de métodos (`.filter().map().reduce()`).

Resultado esperado: `118000` (25000 + 85000 + 8000).

Ejercicio 2 — Validación con Result

Consigna: Implementar un pipeline de validación para datos de un formulario de contacto:

```
type ContactForm = { name: string; email: string; message: string };
```

1. Crear un validator `requireField(fieldName)` usando partial application.
2. Crear un validator `minLength(field, min)` usando currying.
3. Crear `validateContactEmail` que verifique formato de email.
4. Componer los 3 con `chain` y `reduce`.
5. Testear con datos válidos e inválidos.

Ejercicio 3 — Recursión de cola

Consigna en Clojure:

1. Implementar `(product-list nums acc)` que calcule el producto de todos los números usando `recur`.
2. Implementar `(count-if pred items acc)` que cuente cuántos elementos satisfacen el predicado.
3. Trazar paso a paso la ejecución de `(product-list [2 3 4] 1)`.

Consigna en TypeScript:

4. Implementar `productList(nums, acc = 1)` con el mismo patrón de acumulador.
5. ¿Qué pasa si llamas `productList` con un array de 100.000 elementos? ¿Por qué?

Ejercicio 4 — Middleware composable

Consigna:

1. Implementar un middleware `withTimestamp()` que agregue `meta.timestamp` al request.
2. Implementar un middleware `withRequestId()` que agregue `meta.requestId` con un UUID.
3. Componerlos con `pipe` junto con `withAuth` y `withLogging`.
4. Verificar que el handler final recibe un request con todas las transformaciones aplicadas.

Autoevaluación

Respondé estas preguntas antes de hacer el TP:

1. ¿Cuál es la diferencia entre `map` y `flatMap`? ¿Cuándo usarías cada uno?
 2. ¿Qué problema resuelve `Result<T, E>` que `try/catch` no resuelve bien?
 3. ¿Cuál es la diferencia entre partial application y currying? Dá un ejemplo de cada uno.
 4. ¿Por qué `recur` en Clojure no produce stack overflow? ¿Qué pasa en TypeScript/JavaScript?
 5. ¿Cuándo conviene usar `memoize` y cuándo NO?
 6. ¿Qué ventaja tiene expresar reglas de validación como datos (como en el ejemplo de `user-rules` de Clojure)?
 7. Si tenés un pipeline `pipe(f, g, h)` y `g` puede fallar, ¿cómo lo manejarías con `Result`?
-

Glosario

Término	Definición
First-class function	Función que puede asignarse a variables, pasarse como argumento y devolverse como resultado
HOF (Higher-Order Function)	Función que recibe y/o devuelve otra función
Pipeline	Cadena de transformaciones de datos donde la salida de una función es la entrada de la siguiente
<code>pipe</code>	Función que compone de izquierda a derecha
<code>compose</code>	Función que compone de derecha a izquierda
<code>->></code> (thread macro)	Operador de Clojure equivalente a <code>pipe</code>
Partial application	Fijar algunos argumentos de una función para obtener una más específica
Currying	Transformar una función de N argumentos en N funciones de 1 argumento
<code>Result<T, E></code>	Tipo que modela éxito (<code>ok</code>) o error como un valor explícito
Middleware	HOF que recibe un handler y devuelve un handler transformado
Recursión de cola	Recursión donde la llamada recursiva es la última operación (sin trabajo pendiente)
<code>recur</code>	Palabra clave de Clojure que garantiza tail call optimization
Memoization	Cache de resultados de funciones puras para evitar recálculos
Lazy sequence	Secuencia que no calcula sus elementos hasta que se necesitan
Data-driven programming	Patrón donde la lógica se expresa como datos en lugar de código
Inmutabilidad	Principio de no modificar valores existentes, sino crear nuevos
<code>flatMap</code>	Operación que transforma y aplanar un nivel de anidamiento
<code>reduce</code> / <code>fold</code>	Operación que pliega una colección a un valor de cualquier tipo

Referencias

- Filminas de clase: F-01 a F-36 (fuente de verdad).
- Tema 03: Fundamentos de programación funcional (prerrequisito).
- Tema 05: Mónadas en TypeScript — `Result` es la mónada `Either`.
- Tema 06: Efectos y IO — manejo disciplinado de side effects.
- Sweller, J. & Chen, O. (2023). *Extending Cognitive Load Theory* — justifica la estructura de bloques de ≤ 35 min.
- Mayer, R. & Fiorella, L. (2023). *Multimedia Learning Principles* — fundamenta la separación concepto/ejemplo/código.